Bilinear Rank as Proof Cost: Fast Matrix-Multiplication Schemes and Field-Specific Flip Search for zkML

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), matmul track. First version 2026-07-11; revised 2026-07-16 with the measurement program of Appendices B–C (fields × proving stacks × substrates). Companion to the 3×3 *additive-complexity paper (*`doc/matmul_adds_paper.md`*, artifacts DOI 10.5281/zenodo.21240904), the rank-48 rigidity paper, and the flower note. The engine of §5 is* `src/bin/flip23p.rs` *in this repository.*

Abstract

Zero-knowledge machine learning (zkML) — proving that a neural-network inference was computed correctly without re-executing or revealing it — is bottlenecked by the prover, whose work in circuit-based proof systems is counted in **multiplication constraints**: additions and constant-coefficient linear combinations are free. This is the exact inversion of the hardware cost model that keeps fast matrix multiplication out of GPUs, and it makes **bilinear rank** — the number of multiplications in a Strassen-like scheme — the direct cost driver for matrix products between witness values (private weights; attention). Three consequences. (1) Known schemes already pay: recursive rank-48 4×4 blocking proves a 768×768 witness×witness product with ≈4× fewer constraints than the naive circuit, penalty-free because the schemes' additions cost nothing. (2) Coefficient obstructions vanish: schemes over $\mathbb{Q}$ with dyadic (or any rational) coefficients are exact over a prime field, and numerical stability is not a concept. (3) Most interestingly, **rank is field-dependent** — rank 47 for 4×4 exists modulo 2 with no characteristic-0 equivalent known — and zkML fixes a short list of concrete fields (Goldilocks, BabyBear, M31, the BN254/BLS12-381 scalar fields) over which nobody has searched. We port our rational flip-graph engine to $\mathbb{F}_p$ (`flip23p`, Goldilocks first): over a prime field the engine is strictly stronger than its $\mathbb{Q}$ parent — no coefficient growth, no magnitude caps, and every coincidence-solving λ is admissible, so the coplanarity structure that rationality left mostly unusable becomes fully spendable. A verified rank-22 3×3 or rank-47 4×4 over a proof field would be a shippable constraint-count reduction with no analog in the characteristic-0 literature. We state the cost model precisely, including the honest carve-outs (sumcheck/GKR provers multiply matrices in $\sim n^2$ prover work and do not care about rank; public-weight linear layers cost no constraints), and then *measure* it. The measured findings (Appendices B–C): the living operation-count record for rank-48 4×4 is **284**, found in the PLinOpt authors' own distributed artifacts, 57 below their published 341 (checker-verified, Brent-verified); on Groth16/R1CS the predicted constraint counts land exactly but linear-combination *density* charges the rank-48 gadget up to 17.6× — and a PLONKish/AIR port (Plonky3 over BabyBear) **erases that tax structurally**, proving at parity over identical traces; witness generation inherits the exponent advantage with a crossover that is field- *and* substrate-dependent ($n \approx 64$ over Goldilocks-CPU, $n \approx 10^3$ over BabyBear-CPU where 4-lane NEON favors cheap multiplies, moot on the GPU where all algorithms saturate memory bandwidth); and NTT field pricing is measured across BabyBear, Goldilocks, BN254-Fr, and an Apple-GPU column. One law organizes the machine-level results: the winning scheme is decided by the substrate's scarce resource — multiplies, additions, or bytes — and in modern AIR stacks the scarce resource is committed rows, which is exactly what rank reduces. The field-specific rank hunts continue.

1 zkML in one page

A **zero-knowledge proof** (ZKP) lets a prover convince a verifier that a statement holds — here, "this output is what model M computes on input x" — without the verifier re-executing the computation, and optionally without revealing x or M. Two families dominate. **SNARKs** (succinct non-interactive arguments of knowledge; Groth16 [6], PLONK [5]) give constant-or-polylog proof sizes over elliptic-curve-pairing fields

such as the BN254 and BLS12-381 scalar fields (∼254-bit primes). **STARKs** [2] replace pairings with hashes, gain transparency and plausible post-quantum security, and run over small "FFT-friendly" primes chosen for fast number-theoretic transforms — Goldilocks $p = 2^{64} - 2^{32} + 1$ (Plonky2 [14]), BabyBear $p = 2^{31} - 2^{27} + 1$ (RISC Zero [15]), and M31 $= 2^{31} - 1$ (Circle STARKs [8]). (Appendix B explains the FFT/NTT layer these fields are engineered for.)

The asymmetry that defines the field: verification is cheap, **proving is 10^3 to $10^6\times$ the native computation**, all of it exact arithmetic in the proof field. zkML applications — proving a proprietary model was faithfully executed (model privacy), proving an inference inside a blockchain rollup, auditable AI where the checker is cheap — inherit that overhead on every multiply of every layer. Making matrix multiplication cheap *inside a proof* is therefore a different optimization problem from making it fast on silicon, with a different — in fact opposite — cost model.

2 What proving charges

The standard arithmetization, **R1CS** (rank-1 constraint systems, as in Groth16 [6]), expresses a computation as constraints of the form

$$\left(\sum_i a_i w_i\right) \times \left(\sum_i b_i w_i\right) = \left(\sum_i c_i w_i\right)$$

over the proof field, where w is the witness vector and the a_i, b_i, c_i are *constants*. Each constraint carries exactly one multiplication of witness values; the linear combinations inside the parentheses — additions, subtractions, scalings by arbitrary field constants — are free: they are folded into the constraint's constant vectors and cost the prover nothing. PLONK-family systems [5] charge per gate/row with the same shape (custom gates and lookup arguments shift constants around but preserve the headline: **bilinear multiplications of witnesses are the metered resource**).

For an $n \times n$ product $C = A \cdot B$ where both operands are witnesses, each entry $C_{ij} = \sum_k A_{ik} B_{kj}$ is a sum of n witness×witness products; a sum of n products is not one rank-1 quadratic form — it takes n constraints. The naive circuit therefore costs n^3 constraints. Two situations make both operands witnesses in zkML: **private weights** (the commercially central case — the model owner proves inference without revealing the model) and **attention** (QK^T and scores $\cdot\, V$ multiply two activation matrices — witness×witness even when all weights are public). Conversely, a linear layer with *public* weights is a constant linear map and costs zero multiplication constraints; rank optimization has nothing to offer there.

A toy example, 2×2 over $\mathbb{F}_{17}$. Take witnesses $A = \left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$, $B = \left(\begin{smallmatrix} 5 & 6 \\ 7 & 0 \end{smallmatrix}\right)$; then $C = A \cdot B \equiv \left(\begin{smallmatrix} 2 & 6 \\ 9 & 1 \end{smallmatrix}\right) \pmod{17}$. The naive circuit spends one constraint per product — eight constraints of the shape $u_1 = A_{11} \times B_{11}$ — and output sums like $C_{11} = u_1 + u_2$ cost nothing further (they are linear). Strassen's rank-7 scheme spends **seven**: its first product is the single constraint

$$(w_{A_{11}} + w_{A_{22}}) \times (w_{B_{11}} + w_{B_{22}}) = w_{P_1},$$

where all four operand additions sit *inside* the constraint's linear forms, free. There is exactly **one witness per proof**: a single vector w over $\mathbb{F}_{17}$ holding every wire value — a conventional leading 1 (which lets constraints use constants), the inputs, the seven products, the outputs. Concretely, all 20 coordinates:

$$w = (\,1 \mid \underbrace{1,2,3,4}_{A} \mid \underbrace{5,6,7,0}_{B} \mid \underbrace{8,1,6,8,0,5,3}_{P_1..P_7} \mid \underbrace{2,6,9,1}_{C}\,).$$

The subscripted symbols above are coordinates of this one vector, and each constraint is checked by reading them off: for P_1, $(1+4)\cdot(5+0) = 25 \equiv 8$ ✓, and likewise $P_2 = 7 \cdot 5 = 1$, $P_3 = 1 \cdot 6 = 6$, $P_4 = 4 \cdot 2 = 8$, $P_5 = 3{\cdot}0 = 0$, $P_6 = 2{\cdot}11 = 5$, $P_7 = (-2){\cdot}7 = 3$ (all mod 17). The free output combinations reproduce C exactly: $C_{11} = P_1{+}P_4{-}P_5{+}P_7 = 19 \equiv 2$, $C_{12} = P_3{+}P_5 = 6$, $C_{21} = P_2{+}P_4 = 9$, $C_{22} = P_1{-}P_2{+}P_3{+}P_6 = 18 \equiv 1$. (The prover commits to w and proves the constraints hold; the verifier never sees w — that is the "zero-knowledge". Note the naive circuit's witness carries eight product coordinates to Strassen's seven, so lower rank also shortens the committed vector.) Seven constraints instead of eight — 12.5% at one level, compounding to $n^{2.807}$ under recursion — and dyadic coefficients would be equally at home: $\frac{1}{2}$ is just the constant 9 in $\mathbb{F}_{17}$ ($2 \cdot 9 = 18 \equiv 1$).

This is §3's story in miniature; the rank-23 and rank-48 schemes are the same picture with better ratios. (Appendix A runs a complete Setup → Prove → Verify on a two-constraint slice of this example — the whole SNARK pipeline at hand-checkable scale.)

3 Bilinear rank is constraint count

A bilinear scheme of rank r computes an $m \times m$ block product with r products of linear forms. In R1CS each product is exactly one constraint — the linear forms slide into the constraint's free linear parts — so an $m \times m$ witness×witness block product costs r constraints instead of m^3, and recursion compounds it: $T(n) = r\,T(n/m)$, i.e. $n^{\log_m r}$ constraints total, **with zero cost for the combination additions at every level**. The additions that price fast schemes out of silicon are literally free here; there is no numerical-stability tax because arithmetic is exact; and coefficients may be any field constants — dyadic $\frac{1}{2}$'s are simply the constant $(p+1)/2 \bmod p$.

Idealized constraint counts for a witness×witness product (recursion to scalar base; real circuits stop at a base tile, which shifts absolute numbers but not the ordering):

scheme (exponent)	$n = 768$	vs naive	$n = 4096$	vs naive
naive (3.000)	4.53×10^8	1.0×	6.87×10^{10}	1.0×
3×3 : 23 (2.854)	1.72×10^8	2.6×	2.05×10^{10}	3.4×
Strassen 2×2 : 7 (2.807)	1.26×10^8	3.6×	1.39×10^{10}	4.9×
4×4 : 48 **(2.792)**	1.14×10^8	**4.0**×	1.23×10^{10}	**5.6**×
3×3 : 22 (2.814), *if found*	1.31×10^8	3.4×	1.46×10^{10}	4.7×
4×4 : 47 (2.777), *if found*	1.03×10^8	4.4×	1.08×10^{10}	**6.4**×

A commutative carve-in. One refinement the table does not yet fold in: bilinear rank assumes non-commuting entries (that is what makes block recursion valid), but **R1CS witnesses are field elements and commute** — so at the *base tile* of a recursion the true cost is the smaller commutative multiplicative complexity. Rosowski [24] multiplies 3×3 in **21** products over a commutative ring (vs bilinear 23; Makarov's earlier 22), and in general odd-n tiles cost $n(n^2+2n-1)/2$: 5×5 in 85 (vs 93), 7×7 in 217 (vs 250). Blocks do not commute, so interior levels must stay bilinear — a base-tile lever only (and 2×2 gains nothing: 7 is optimal even commutatively) — but any recursion bottoming on 3×3 tiles saves $21/23 \approx 8.7\%$ of its *entire* constraint count; the optimal tile choice is a small dynamic program we leave as a refinement. Flip-graph search has recently been extended to commutative schemes [25], so §5's discovery machinery has a commutative analogue if the base-tile hunt warrants it. Quantified against Appendix B.5's bilinear wins: stopping the rank-48 recursion at an odd tile t and paying Rosowski's $R(t)$ multiplies the total by $R(t)/t^{2.792}$ — 0.977 at t=3, 0.950 at t=5, **0.947 at** t=7 (the optimum) — so today's commutative ceiling is a ≈5.3% saving, smaller than a rank-47's 13.5% or a rank-21's 19%, for the structural reason one should expect: a bilinear record improves the *exponent* and compounds through every level, while commutativity improves one *constant* at the bottom. It stacks multiplicatively with any bilinear win, costs no new mathematics, and — the pricing surprise — is extremely sensitive at the smallest tile: each -1 on the 3×3 commutative count is worth 4.65% of the entire pipeline ($3^{2.792}$ is small), so a 3×3 commutative scheme with 20 products would deliver 7.0% and with 19 products 11.6% — rank-47-class wins from a search space minuscule beside the bilinear ones, with known lower bounds leaving room in the mid-teens. We know of no deployed ZK system exploiting fast multiplication at all — circuit-based practice proves matmul naively, and the systems that escape n^3 do so by *verification* (Freivalds-style random projection, the sumcheck line of §4), not by faster multiplication — so both levers, the exponent and the tile, are currently unshipped.

Two readings. First, **the wins in the top half are available today**: the rank-48 scheme [11, 10] with dyadic coefficients drops into any R1CS/PLONK matmul gadget as-is — and "works over $\mathbb{F}_p$" is not a conjecture: our engines load the scheme and verify all 4096 Brent equations exactly over Goldilocks, BabyBear, and M31 (§5; the dyadic $\frac{1}{2}$'s are the field constants $(p+1)/2$ etc.). Only characteristic 2 excludes it. (Our repository carries the verified scheme and its minimized linear networks.) Second, the *marginal* rows show why new records matter here more than in silicon: each rank step is a few percent per recursion level, compounding — and unlike hardware, nothing eats the margin.

4 The honest carve-outs

Rank is **not** the lever everywhere:

- **Sumcheck/GKR provers do matmul in $\sim n^2$ prover field-ops** (Thaler [4]; GKR [7]); zkCNN [9] and successors build dedicated zkML provers on this line, and where such a prover applies, bilinear rank is irrelevant. The rank lever applies to the (large) circuit-based world: general-purpose SNARK toolchains, zkVMs executing matmul as arithmetic, PLONK/R1CS pipelines such as EZKL [13], and folding-based IVC (Nova [16]) whose per-step cost is the step circuit's constraint count.
- **Public-weight linear layers are free**; rank helps witness×witness products only — private weights and attention.
- **Constraint count is not wall-clock**: prover time also includes commitments (multi-scalar multiplications / hashing); fewer constraints shrink those too, but constants differ by system. One further assumption our ratios inherit: "additions are free" holds for the constraint *count*, but deep recursion densifies the R1CS rows — each level multiplies the nonzero coefficients in the constraints' linear forms, and the prover's linear-algebra phase (computing $A \cdot w$, $B \cdot w$, $C \cdot w$ and the witness itself) scales with that density. This is exactly the lever our companion additions work pulls: low-adds schemes (the 55-add 3×3 networks; the DPS 341-op rank-48 networks) bound the densification rate, so rank and adds are dual levers — rank sets how many constraints, adds set how dense they are.
- **Nonlinearities dominate some workloads** (range checks, lookups for ReLU/quantization); the matmul share is large in attention/convolution-heavy models, smaller in lookup-heavy quantized pipelines.

5 Field-specific rank: the open frontier, and flip23p

Bilinear rank depends on the coefficient field. The emblem: AlphaTensor's rank-**47** 4×4 scheme exists over $\mathbb{F}_2$ [3], no characteristic-0 rank-47 is known, and our companion work proved the only $\mathbb{Q}$-usable rank-48 class *rigid* under a large certified move system — evidence that characteristic 0 is genuinely constrained in ways a fixed finite field need not be. zkML nominates a short, concrete list of fields that matter commercially — Goldilocks, BabyBear, M31, BN254-Fr, BLS12-381-Fr — and, to our knowledge, **no rank search has ever been run over any of them**: the flip-graph literature works over $\mathbb{F}_2$ and small extensions [17], the explicit records over $\mathbb{Z}/\mathbb{Q}$. A verified 3×3 rank-22 — or 4×4 rank-47 — over Goldilocks would be a new kind of record with an immediate consumer: 2 to 4% fewer constraints per recursion level in deployed proof systems, compounding as in §3.

`flip23p` is our rational flip engine (splits, λ-flips, reductions, canonical gauge, exact Brent verification — the machinery of the companion papers) re-based onto $\mathbb{F}_p$, Goldilocks first. Over a prime field the engine is *strictly stronger* than its $\mathbb{Q}$ parent:

1. **No coefficient growth.** $\mathbb{Q}$-walks need magnitude caps and dyadic bookkeeping; field elements don't grow. Every cap in the $\mathbb{Q}$ engine is deleted, not loosened.
2. **Every solved flip is admissible.** Over $\mathbb{Q}$, a coincidence ($f_i + \lambda f_j \propto f_m$, the Cramer-solved move that makes flips productive) is usable only when λ is dyadic — the filter discarded most solutions. Over $\mathbb{F}_p$ every nondegenerate solution is a legal move: the coplanarity structure ("H4's raw material" in the flower note) becomes fully spendable.
3. **The gauge simplifies**: monic normalization (leading coefficient 1, scalars folded into the c-slot); proportional $\Leftrightarrow$ equal, with none of $\mathbb{Q}$'s content/sign bookkeeping.

First measurements (2026-07-11; §8): the 55-addition record scheme loads and verifies over Goldilocks (729 Brent equations mod p); the census gate matches the $\mathbb{Q}$ engine exactly (9 shared pairs, weight 177, distinct-rows 40). One honest surprise: the seed's solved-move silence is *not* the dyadic filter's fault — even with all field λ available, its 9 shared pairs admit no coincidence solution; the coplanarity geometry genuinely fails there. Mobility arrives at once elsewhere: a 25-second smoke storm from the most mobile census seed executed 1.3M moves with 369K solved flips and ∼10K reductions — a far higher solved-flip fraction than the $\mathbb{Q}$ storm

ever achieved, the freed λ paying immediately. No sub-23 rank has appeared in smoke tests (nor was one expected at that effort); the campaigns are the point.

6 What transfers from the existing program

- **Engines and discipline.** flip48/flip23's storm, closure, census, novelty and thin-storm machinery ports mechanically (v1 of `flip23p` carries storm/census/closures; the beam-chase and exhaustive campaign modes follow). The operational rules hard-won in the companion papers — completion logs, full-frontier dumps, control baselines, budget caps — apply verbatim.
- **Seeds.** Every scheme we hold is a valid $\mathbb{F}_p$ scheme: the 17,376 database classes, our 53 lifted classes, the 46 storm-new classes, and the 467 mod-2-invisible dyadic $\mathbb{Q}$-schemes (denominators are field constants). The mobility census that ranked $\mathbb{Q}$-seeds re-runs unchanged.
- **Verified gadgets.** The Lean pipeline that certified the 55-add scheme extends naturally: a scheme-to-circuit emitter (circom / Halo2 templates for the 23- and 48-multiplication blocks) with a machine-checked correctness certificate is a deliverable the proof-systems world actually values.
- **What does not transfer:** the additive-complexity results (55 adds, the no-54 theorem). Additions are free here; that program's value stays in classical hardware and in witness-generation speed.

7 Program

(1) Goldilocks campaigns: storm + closure portfolios over the census-ranked seeds; targets, in order of plausibility $\times$ value: any new-class harvest over $\mathbb{F}_p$, rank 22 at 3×3, rank 47 at 4×4 (the 4×4 port is the same edit at 16 coordinates). (2) Small-field variants: BabyBear and M31 (31-bit arithmetic, $\sim$4$\times$ faster walks), then BN254-Fr. (3) Rectangular tiles: transformer shapes want $\langle m,n,k\rangle$ records; the engine generalizes as flip48 $\rightarrow$ flip23 did. (4) Verified gadget emitter + benchmark note: *largely done* — Appendix C now carries measured constraint counts and prover times on arkworks Groth16 (`benchzk`) and a Plonky3 AIR gate (`benchair`), with the witness-generation and NTT measurement program beside them; Lean-certified gadget templates remain open. The next deployment-shaped artifact is a memory-mediated zkVM precompile chip, where the rank-48 row reduction becomes prover wall-clock.

8 Reproduction

```
cargo build --release --bin flip23p
./target/release/flip23p --census --dir matmul/mm23      # gates
./target/release/flip23p --dir matmul/seeds23/13a5bd4n \
    --seconds 25 --threads 4 --out matmul/found23p       # smoke storm
# any verified rank <= 22 over Goldilocks lands in
# found23p/RECORDP_rank*.txt and is announced loudly

# measurement program (Appendices B-C); each binary gates itself
# (checker / Brent / bit-for-bit reference) before timing
(cd matmul/benchzk && cargo run --release -- 64 rank48 --matlv 2 --full)
(cd matmul/benchair && cargo run --release)            # AIR gate A/B
(cd matmul/benchmetal && cargo run --release)          # GPU tiles
(cd matmul/benchmetal && cargo run --release --bin benchntt_metal)
cargo run --release --bin benchntt_g                   # Goldilocks NTT
cargo run --release --bin benchntt_bb                  # BabyBear NTT
cargo run --release --bin bench284r                    # witness-gen A/B
cargo run --release --bin bench_bb                     # NEON lanes
```

Appendix A. Setup, Prove, Verify — a complete toy proof

This appendix runs the entire SNARK pipeline, Groth16-shaped, on a two-constraint slice of §2's example, every number over $\mathbb{F}_{17}$. One honest simplification, flagged where it occurs: real systems wrap the values below

in elliptic-curve encodings ("in the exponent"), which is what makes hiding cryptographic; the *arithmetic* — R1CS → QAP → divisibility check at a secret point — is shown exactly as real provers compute it.

The statement. For reference, the full toy matrices of §2, whose entries supply every private value below:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 0 \end{pmatrix}, \qquad C = A \cdot B \equiv \begin{pmatrix} 2 & 6 \\ 9 & 1 \end{pmatrix} \pmod{17}.$$

The prover claims: "I know private values $A_{11}, A_{22}, B_{11}, B_{12}, B_{22}$ (here $1, 4, 5, 6, 0$ — the marked entries of A and B) such that $P_1 = (A_{11}+A_{22})(B_{11}+B_{22}) = 8$ and $P_3 = A_{11}(B_{12}-B_{22}) = 6$." The claimed products $(8, 6)$ are **public**; the five inputs stay private. The witness (public part first):

$$w = (\,1,\ P_1{=}8,\ P_3{=}6 \mid A_{11}{=}1,\ A_{22}{=}4,\ B_{11}{=}5,\ B_{12}{=}6,\ B_{22}{=}0\,)$$

(Why no A_{12}, A_{21}, B_{21} here, when §2's full witness carries every entry? Because this appendix proves only the two-constraint slice: a witness coordinate earns its place by being *wired into some constraint*, and c_1, c_2 below never touch those entries. Including them would just add all-zero rows to the QAP table — their coefficient polynomials would be identically 0 on all three sides. In the full 7-constraint Strassen proof all eight inputs appear, since e.g. $M_5 = (A_{11} + A_{12})B_{22}$ wires in A_{12}. Note the contrast: $B_{22} = 0$ *is* present despite its value being zero — membership is about the circuit's wiring, not the value.)

The two constraints (check: $5 \cdot 5 = 25 \equiv 8$, $1 \cdot 6 = 6$):

$$c_1 : (w_{A_{11}} + w_{A_{22}}) \times (w_{B_{11}} + w_{B_{22}}) = w_{P_1}$$
$$c_2 : (w_{A_{11}}) \times (w_{B_{12}} - w_{B_{22}}) = w_{P_3}$$

Step 0 — constraints become one polynomial identity (the QAP). QAP stands for **Quadratic Arithmetic Program** [21] — the standard compilation of an R1CS into a single polynomial divisibility test, so that "all constraints hold" can later be checked at *one point* instead of once per constraint. Three sub-steps.

(a) Read off each constraint's coefficients. A witness **coordinate** is one entry of the vector w (such as $w_{A_{11}}$); each constraint assigns every coordinate three constant **coefficients** — its multiplier in the left factor, the right factor, and the output side. Written out in full:

$$\begin{aligned} c_1 :&\ \text{left} = 1\,w_{A_{11}} + 1\,w_{A_{22}} \quad \text{(every unlisted coordinate: 0)} \\ &\ \text{right} = 1\,w_{B_{11}} + 1\,w_{B_{22}}, \qquad \text{out} = 1\,w_{P_1} \\ c_2 :&\ \text{left} = 1\,w_{A_{11}}, \qquad \text{right} = 1\,w_{B_{12}} + (-1)\,w_{B_{22}}, \qquad \text{out} = 1\,w_{P_3} \end{aligned}$$

(b) Build the Lagrange interpolation basis. Assign constraint c_j to the evaluation point $x = j$. The **Lagrange basis polynomial** L_j is the unique lowest-degree polynomial equal to 1 at its own point and 0 at every other; the general formula $L_j(x) = \prod_{k \neq j} \frac{x-k}{j-k}$ gives, for our points $\{1, 2\}$,

$$L_1(x) = \frac{x-2}{1-2} = 2 - x, \qquad L_2(x) = \frac{x-1}{2-1} = x - 1,$$

with $L_1(1) = 1, L_1(2) = 0$ and $L_2(1) = 0, L_2(2) = 1$ (over $\mathbb{F}_{17}$ the division by $1 - 2 = -1$ is multiplication by 16 — every nonzero denominator is invertible in a field). These are building blocks: the polynomial taking values (v_1, v_2) at $(1, 2)$ is exactly $v_1 L_1 + v_2 L_2$.

(c) Interpolate each coordinate's coefficients. For coordinate k, collect its left-factor coefficients across constraints as the value list (value in c_1, value in c_2) and set $A_k(x) = (\text{coeff in } c_1)\,L_1(x) + (\text{coeff in } c_2)\,L_2(x)$; likewise B_k from right-factor and C_k from output coefficients. Two worked rows: A_{22} has left-coefficients $(1, 0)$, so its A-poly is $1 \cdot L_1 + 0 \cdot L_2 = 2 - x$; B_{22} has right-coefficients $(1, -1)$, so its B-poly is $L_1 - L_2 = (2 - x) - (x - 1) = 3 - 2x$. The full table:

coord	A-side of	A-poly	B-side of	B-poly	C-side of	C-poly
A_{11}	c_1, c_2	1	—	0	—	0
A_{22}	c_1	$2-x$	—	0	—	0
B_{11}	—	0	c_1	$2-x$	—	0
B_{12}	—	0	c_2	$x-1$	—	0
B_{22}	—	0	$c_1(+)$, $c_2(-)$	$3-2x$	—	0
P_1	—	0	—	0	c_1	$2-x$
P_3	—	0	—	0	c_2	$x-1$

Weight each polynomial by its witness value and sum:

$$A(x) = 1\cdot 1 + 4(2-x) = 9-4x, \qquad B(x) = 5(2-x)+6(x-1) = x+4, \qquad C(x) = 8(2-x)+6(x-1) = 10-2x.$$

Sanity: at x=1: $A\cdot B = 25 \equiv 8 = C$ (c_1 ✓); at x=2: $A\cdot B = 6 = C$ (c_2 ✓). **Both constraints hold** $\iff$ $A(x)B(x)-C(x)$ **vanishes at** $x = 1, 2$ $\iff$ **it is divisible by** $Z(x) = (x-1)(x-2)$**.** Over $\mathbb{F}_{17}$:

$$P(x) = AB - C = -4x^2 - 5x + 26 \equiv 13x^2 + 12x + 9, \qquad Z(x) = x^2 - 3x + 2, \qquad H(x) = P/Z = 13$$

(exact division, remainder 0).

Where Z comes from. Z is the **vanishing polynomial** of the constraint points. Constraint c_j holds exactly when $P(j) = A(j)B(j) - C(j) = 0$ — that is, when $(x-j)$ divides P. Both constraints hold exactly when $(x-1)$ and $(x-2)$ *both* divide P, i.e. when their product $Z(x) = (x-1)(x-2) = x^2 - 3x + 2$ does. (With m constraints, $Z = \prod_{j=1}^{m}(x-j)$: one root per constraint — "P vanishes wherever a constraint lives.")

How $H = 13$ was computed. Ordinary polynomial division over $\mathbb{F}_{17}$, which here collapses to one step: P and Z both have degree 2, so H is the constant (leading coefficient of P) $\div$ (leading coefficient of Z) $= 13 \div 1 = 13$, and exactness is one multiplication:

$$13\,Z = 13x^2 + 13(-3)x + 13\cdot 2 = 13x^2 - 39x + 26 \equiv 13x^2 + 12x + 9 \pmod{17} = P \quad \text{(remainder 0). } \checkmark$$

The exactness is not an algebraic accident; it is the entire content of the proof: **division comes out clean precisely because the witness satisfies both constraints**. Tamper with the claim — say the prover asserts $P_1 = 9$ instead of 8 — and $C(x)$ becomes $9(2-x)+6(x-1)$, so $P(1) = 25 - 9 = 16 \neq 0$: now $(x-1)$ no longer divides P, no valid H exists at all, and the cheater's only remaining hope is faking the divisibility check at the one hidden point τ — exactly what Step 3's Schwartz–Zippel argument makes overwhelmingly unlikely.

The secret knowledge is compressed into: "I can exhibit witness-weighted A, B, C and a quotient H with $AB - C = HZ$."

Step 1 — Setup (the trusted ceremony). Sample a secret point, say $\boldsymbol{\tau = 5}$ — the "toxic waste" — and publish the powers of τ and the QAP polynomials evaluated at τ, each wrapped in an encoding enc$(\cdot)$ that supports additions and scalings but hides the value — defined concretely just below. Then **destroy** τ: anyone holding it can forge proofs — hence the multi-party ceremonies around Groth16, and the transparent (no-τ) alternatives like STARKs. For our toy, the published material (the **common reference string**, CRS) is *witness-independent* — one ceremony serves every future proof of this circuit — and consists of the encoded evaluations at $\tau = 5$ of each *coordinate* polynomial from Step 0's table, plus $Z(5)$ and encodings of the powers of τ that H may need (here just τ^0, since our H has degree 0). Spelled out:

$$\begin{array}{lll} A_{A_{11}}(5) = 1 & A_{A_{22}}(5) = 2 - 5 \equiv 14 & \\ B_{B_{11}}(5) \equiv 14 & B_{B_{12}}(5) = 4 & B_{B_{22}}(5) = 3 - 10 \equiv 10 \\ C_{P_1}(5) \equiv 14 & C_{P_3}(5) = 4 & Z(5) = 4\cdot 3 = 12 \end{array}$$

(every unlisted coordinate's polynomial is identically 0). Note what is **not** published: anything witness-dependent — $A(x)$, $B(x)$, $C(x)$, $P(x)$, $H(x)$ belong to the prover, not the setup.

What enc(·) **is.** Exponentiation in a group where the reverse direction — recovering the exponent, the *discrete logarithm* — is computationally infeasible: fix a public generator g and set $\mathrm{enc}(v) = g^v$. Real systems use elliptic-curve groups of size $\sim 2^{256}$; the toy admits an honest miniature. Inside the integers mod 103 (a prime), the powers of $g = 72$ form a subgroup of order exactly 17 — $72^{17} \equiv 1 \pmod{103}$, no smaller power is 1 — so exponents live precisely in $\mathbb{F}_{17}$:

$$\mathrm{enc}(v) = 72^v \bmod 103: \qquad \begin{array}{l} \mathrm{enc}(0) = 1,\ \mathrm{enc}(1) = 72,\ \mathrm{enc}(4) = 23,\ \mathrm{enc}(6) = 61,\ \mathrm{enc}(9) = 81, \\ \mathrm{enc}(10) = 64,\ \mathrm{enc}(12) = 13,\ \mathrm{enc}(13) = 9,\ \mathrm{enc}(14) = 30. \end{array}$$

What Setup *literally* publishes is this column: $\mathrm{enc}(1) = 72$, $\mathrm{enc}(14) = 30$, $\mathrm{enc}(4) = 23$, $\mathrm{enc}(10) = 64$ for the coordinate polynomials, $\mathrm{enc}(12) = 13$ for $Z(5)$ — the raw values displayed above were narrator's courtesy. (That $\mathrm{enc}(12) = 13$ numerically equals H's value is pure coincidence.) Two properties do all the work: (i) **hiding** — recovering v from $72^v \bmod 103$ is the discrete-log problem, brute-forceable at order 17 (the toy's one dishonesty: 17 candidates) but infeasible at order $\sim 2^{256}$, which is why publishing encodings of powers of τ does not reveal τ; (ii) **linearity passes through** — $\mathrm{enc}(a)\,\mathrm{enc}(b) = g^{a+b} = \mathrm{enc}(a+b)$ and $\mathrm{enc}(v)^c = \mathrm{enc}(cv)$ for known c, so anyone can form known-coefficient linear combinations of hidden values without decoding. What the encoding can *never* do is multiply two hidden values; that asymmetry is the crypto-layer origin of this paper's cost model — linear operations ride through for free, each hidden×hidden product must be paid for as a constraint, and the verifier's one pairing (Step 3) grants exactly one such product, just enough for the final check.

Step 2 — Prove. The prover, holding w, computes the *encoded* evaluations — each a **linear combination** of published encodings with witness coefficients (this is why R1CS's free-linear-combination structure is the whole design):

$$\begin{aligned} &A(5) = w_{A_{11}} A_{A_{11}}(5) + w_{A_{22}} A_{A_{22}}(5) = 1 \cdot 1 + 4 \cdot 14 = 57 \equiv 6, \\ &B(5) = 5 \cdot 14 + 6 \cdot 4 + 0 \cdot 10 = 94 \equiv 9, \qquad C(5) = 8 \cdot 14 + 6 \cdot 4 = 136 \equiv 0, \qquad H(5) = 13. \end{aligned}$$

Cross-check by direct evaluation (which only we, the omniscient narrators, can do): $A(x) = 9 - 4x$ at 5 gives $-11 \equiv 6$ ✓, $B = x + 4$ gives 9 ✓, $C = 10 - 2x$ gives 0 ✓ — the same numbers, but the prover's route touched only published encodings and witness values, never τ itself. In encoded form — all a real prover ever holds — the same combinations run *inside the group*, by property (ii):

$$\begin{aligned} &\mathrm{enc}(A(5)) = \mathrm{enc}(A_{A_{11}}(5))^{w_{A_{11}}}\,\mathrm{enc}(A_{A_{22}}(5))^{w_{A_{22}}} = 72^1 \cdot 30^4 = 72 \cdot 8 \equiv 61 \pmod{103} = \mathrm{enc}(6), \\ &\mathrm{enc}(B(5)) = 30^5\,23^6\,64^0 \equiv 81 = \mathrm{enc}(9), \qquad \mathrm{enc}(C(5)) = 30^8\,23^6 \equiv 1 = \mathrm{enc}(0), \\ &\mathrm{enc}(H(5)) = \mathrm{enc}(\tau^0)^{13} = 72^{13} \equiv 9 = \mathrm{enc}(13). \end{aligned}$$

The transmitted proof is literally those four group elements, $\boldsymbol{\pi = (61, 81, 1, 9)}$, and nothing else. (Both C-coordinates here happen to be public outputs, so the verifier could rebuild $\mathrm{enc}(C(5))$ itself; in general the proof carries the private part C_{priv}.) In Groth16, after optimizations, the proof compresses to three curve points, ∼200 bytes, *independent of circuit size*. The prover never learns τ.

Step 3 — Verify. The verifier reconstructs the *public* part of $C(\tau)$ itself from the claimed outputs $(8, 6)$ — the prover cannot lie about what is being claimed — then checks **one multiplicative relation**. In real systems this is one *pairing* equation: a pairing satisfies $e(g^a, g^b) = e(g, g)^{ab}$, multiplying two hidden values exactly once — the single multiplication the encoding ever grants. Our mod-103 toy group has no pairing, so here (the toy's one honest deviation) we decode and check in the clear:

$$A(\tau)B(\tau) - C(\tau) \stackrel{?}{=} H(\tau)Z(\tau): \qquad 6 \cdot 9 - 0 = 54 \equiv 3, \qquad 13 \cdot 12 = 156 \equiv 3. \quad ✓$$

Why this convinces (soundness). A cheating prover without a valid witness needs $AB - C$ divisible by Z; the best it can do is fake the relation at the single hidden point τ. Two distinct low-degree polynomials agree at a random point with probability $\leq \deg / |\mathbb{F}|$ — about 2/17 in the toy (which is why $\mathbb{F}_{17}$ is a classroom field), about 2^{-250} over a real 254-bit field. Not knowing τ, the prover cannot aim: Schwartz–Zippel is the heart of the construction.

Why it reveals nothing (zero-knowledge). The verifier sees only encodings, never witness coordinates. (Full Groth16 additionally randomizes each proof with masking multiples of $Z(x)$ so two proofs of the same witness look independent; omitted for clarity.)

The connection to rank, one last time. Each constraint became one interpolation point, one row of the QAP table, one share of prover work and setup size. Prove a witness×witness matrix product with a rank-r scheme and this whole pipeline is r constraints per block instead of m^3 — the scheme's additions never appear as constraints at any stage; they live inside the linear combinations, which the encodings support for free.

Appendix B. The FFT/NTT layer — how the polynomial pipeline scales

Appendix A ran the whole pipeline at two constraints, where every polynomial step was hand-arithmetic: interpolate from two points, one product, one division. A real circuit has *millions* of constraints, and each of those steps done naively costs $O(m^2)$ field operations. The workhorse that makes them $O(m \log m)$ is the fast Fourier transform run over $\mathbb{F}_p$ — the **NTT** (number-theoretic transform). This appendix shows it working at toy scale in the same $\mathbb{F}_{17}$, then gives the production numbers.

B.1 Where transforms appear. Four steps of Appendix A become transform calls at scale: (1) *interpolation* (Step 0): witness columns → the coefficient polynomials A, B, C — one **inverse NTT** each replaces the $O(m^2)$ Lagrange formulas; (2) *products* such as $A(x)B(x)$: NTT both onto a larger domain, multiply **pointwise**, inverse-NTT back; (3) *the quotient* $H = P/Z$: pointwise division on a coset — see B.3, where this becomes almost comically cheap; (4) (STARKs) the *low-degree extension*: re-evaluate the trace on a 2 to 8× larger domain — Reed–Solomon encoding, again NTTs — before Merkle-hashing and FRI.

B.2 The transform itself, at size 4 in $\mathbb{F}_{17}$. The classical FFT evaluates a polynomial at the complex n-th roots of unity $e^{2\pi ik/n}$; the sines and cosines are merely how $\mathbb{C}$ *parametrizes* its roots of unity. The algorithm needs only three algebraic facts: $\omega^n = 1$, $\omega^{n/2} = -1$, and "squaring the n-th roots gives the $(n/2)$-th roots" (what lets the problem halve recursively). These are radix-2 requirements, so n is always a power of two — circuits pad to the next 2^k with dummy constraints rather than ever running an odd-length transform (mod 17 odd orders don't even exist: every element order divides 16). Any field with an element of order n runs the identical recursion — exactly, with no rounding. In $\mathbb{F}_{17}$, $p - 1 = 16$, so orders up to 16 exist; $\omega = 4$ has order 4:

$$\omega = 4: \quad \omega^1 = 4,\ \omega^2 = 16 \equiv -1,\ \omega^3 = 13,\ \omega^4 = 1; \qquad D = \{1, 4, 16, 13\}.$$

Run it on Appendix A's own $P(x) = 13x^2 + 12x + 9$. Split by even/odd coefficients — $P(x) = E(x^2) + x\,O(x^2)$ with $E(y) = 9 + 13y$, $O(y) = 12$. Now watch the domain collapse: the four points square pairwise onto just two values, $1^2 = 1$, $4^2 = 16$, $16^2 = 256 \equiv 1$, $13^2 = 169 \equiv 16$ — so E and O need evaluating only at $\{1, 16\} = \{1, -1\}$. That collapse *is* the halving. Combine with the butterfly $P(x) = E(x^2) + x\,O(x^2)$ at each of the four x:

$$E(1) = 22 \equiv 5, \qquad E(16) = 9 + 208 \equiv 13, \qquad O \equiv 12 \text{ everywhere;}$$
$$P(1) = 5 + 1 \cdot 12 \equiv 0, \quad P(4) = 13 + 4 \cdot 12 \equiv 10, \quad P(16) = 5 + 16 \cdot 12 \equiv 10, \quad P(13) = 13 + 13 \cdot 12 \equiv 16.$$

Six multiplications instead of the naive twelve (naively each point costs three: $x \cdot x$, $13 \cdot x^2$, $12 \cdot x$). At this size even the six flatter to deceive — $13 \cdot 1$ and $1 \cdot 12$ are multiplications by 1, and since $16 \equiv -1$, $13 \equiv -4$, two more are negations or reuses — but that is exactly the right lesson: the honest claim is asymptotic. The recursion satisfies $T(n) = 2\,T(n/2) + O(n) = O(n \log n)$ against $O(n^2)$: invisible at $n = 4$, a $\sim 10^5\times$ factor at $n = 2^{20}$. (Note $P(1) \equiv 0$: x=1 is one of Appendix A's constraint points, and the transform *displays* constraint c_1 holding. Real systems put **all** constraints on such a domain, so the whole R1CS check is visible in one spectrum.) The inverse transform is the same butterfly with $\omega^{-1} = 13$ and a global factor $n^{-1} = 4^{-1} \equiv 13$.

B.3 Why roots of unity: the vanishing polynomial collapses. Appendix A built $Z(x) = (x-1)(x-2)$ by multiplying linear factors — fine at 2 points, hopeless at 2^{20}. On the domain D above (we write D for the domain — H is already taken by the quotient polynomial; much of the SNARK literature does the opposite, H for the domain and $h(x)$ for the quotient),

$$Z_D(x) = x^4 - 1 \qquad \text{(generally } x^n - 1\text{)},$$

one subtraction to evaluate anywhere. Better still, the quotient $H = P/Z$ is computed on a *shifted coset* $g \cdot D$ where Z never vanishes — and there it is not merely cheap but **constant**: on $3D = \{3, 12, 14, 5\}$ every point satisfies $x^4 = 3^4 \equiv 13$, so $Z_D \equiv 12$ across the entire coset, and "divide by Z" is one multiplication by $12^{-1} \equiv 10$. Appendix A's polynomial long division becomes: NTT P onto the coset, scale by a constant, inverse NTT.

B.4 Two-adicity — why the proof fields look the way they do. A radix-2 NTT of size 2^k needs an element of order 2^k, i.e. $2^k \mid p-1$. That single divisibility requirement shapes the field zoo:

field	p	$p-1$ factors as	max radix-2 NTT
toy $\mathbb{F}_{17}$	17	2^4	16
Goldilocks	$2^{64}-2^{32}+1$	$2^{32}(2^{32}-1)$	2^{32}
BabyBear	$2^{31}-2^{27}+1$	$2^{27} \cdot 3 \cdot 5$	2^{27}
BN254-Fr	$\sim 2^{254}$	$2^{28} \cdot$ (odd)	2^{28}
M31	$2^{31}-1$	$2 \cdot 3^2 \cdot 7 \cdot 11 \cdot 31 \cdot 151 \cdot 331$	**2**

M31 is the cautionary tale: two-adicity 1, no radix-2 domains at all — the reason Circle STARKs [8] exist (they run the transform on the unit circle $x^2 + y^2 = 1$ over $\mathbb{F}_p$, which has 2^{31} points with perfect 2-adic structure). Two further design notes. *Exactness*: a floating-point FFT's rounding is harmless in signal processing and fatal here — one wrong field element breaks the divisibility identity — while the NTT is exact by construction. *Reduction cost*: the NTT does one modular reduction per multiply, so proof fields are engineered for cheap reduction (Goldilocks reduces with shifts and adds; BabyBear/M31 fit 32-bit lanes and vectorize).

B.5 Production scale. Order-of-magnitude anchors (2026 practice). A single proof segment typically carries 2^{20} to 2^{24} constraints or trace rows ($\approx 10^6$ to 1.6×10^7); STARK blowups of 2 to 8× put the largest NTTs at 2^{23} to 2^{27} points — BabyBear's 2^{27} ceiling is not an accident but the binding constraint (a 2^{24}-row trace at blowup 8 uses the whole two-adic budget). A Groth16 prover runs ≈ 7 size-m FFTs plus 4 size-m multi-scalar multiplications; STARK provers are NTT + hashing dominated. Computations bigger than one segment shard into thousands of segments whose proofs are aggregated by recursion. §3's zkML numbers land here: one $n = 4096$ witness×witness product costs 1.23×10^{10} constraints under rank-48 recursion — $\approx 10^4$ segments of 2^{20} — versus 6.87×10^{10} naive, a 5.6× cut in segments, NTTs, and commitments alike; per-layer transformer matrices (2048 to 8192 on a side) sit squarely in the §3 table's range. The division of labor is exact: the NTT/commitment layer fixes the **cost per constraint**, and the bilinear rank of §3 fixes **how many constraints there are**. Lower rank does not speed up the transform; it shrinks the transform.

Measured on this machine (Apple M-series, 10 P-cores). The field-choice folklore, quantified: the same radix-2 transform, one core, correctness gated (root orders, inverse round-trip, naive-DFT cross-check at $n = 8$, polynomial product vs schoolbook):

domain	BB, GPU (M4 Pro)	BB, 1T	Goldilocks, 1T	BN254-Fr, 1T	BN254-Fr, 6T
2^{14}	0.07 ms	0.3 ms	1.1 ms	2.8 ms	2.9 ms
2^{18}	0.17 ms	5.7 ms	34 ms	55 ms	74 ms
2^{22}	3.0 ms	0.118 s	0.68 s	1.11 s	0.23 s
2^{25}	44 ms	1.15 s	7.8 s	—	2.4 s

The GPU column (Metal compute kernels, gated bit-for-bit against the CPU implementation, forward and round-trip) is the same transform 26–39× faster at proving-relevant domains — and a bandwidth audit shows why the substrate law holds here too: at 2^{25} the 25 butterfly stages move ≈ 12.8 GB in 44 ms, ~ 290 GB/s,

saturating the chip. Small domains are launch-bound (72 µs at 2^{14} is mostly encoder overhead — the measured reason production GPU provers batch many small NTTs into single kernels). Small fields win the transform per core, and by more than word width alone: BabyBear (Montgomery, two-adicity 2^{27}, generator 31) runs 5.8–6.8× faster than Goldilocks — 32-bit multiplies *and* a halved cache footprint compound — and ≈9.4× faster than single-thread BN254-Fr at 2^{22}. Goldilocks' own per-core advantage over BN254-Fr is 1.6–2.6× (largest while the working set fits in cache); the 254-bit field claws wall clock back only through arkworks' parallel FFT (4.7× at 2^{22}, and *slower* than its own single thread at mid-size domains where fork/join overhead dominates) — parallelism the small-field implementations could match. Measured under concurrent background load; the ratios, not the absolutes, are the payload.

What a new record would buy. Prover time is ≈ NTT ($m \log m$) + commitments (m), so speedup tracks the constraint ratio (the log factor barely moves). At $n = 4096$, folding in §3's exponents:

- a **rank-47** 4×4 is a drop-in upgrade of today's best base: $(48/47)^6 \approx \mathbf{1.13}\times$ fewer matmul constraints (13.5%) — at a 90% matmul share, ≈12% end-to-end prover time;
- a **rank-22** 3×3 speeds 3×3-blocked pipelines by $\mathbf{1.40}\times$ but still trails the rank-48 base (0.84×) — its value is scientific (first sub-23) and field-cartographic, not operational;
- a **rank-21** 3×3 — the Strassen-beating line — would take over as the best known base outright: $\mathbf{1.99}\times$ over rank-23 pipelines and $\mathbf{1.19}\times$ over rank-48, i.e. ≈19% fewer constraints, NTTs, and committed elements than anything available today.

End-to-end wall clock scales by the circuit's matmul share (80 to 95% in typical zkML inference), Amdahl-style.

Larger bases (records as of July 2026). The $\mathbb{F}_p$-valid records above 4×4 — mod-2 results do *not* transfer to odd proof primes — are: 5×5: **93** and 6×6: **153** (Moosbauer–Poole [22], explicitly "over arbitrary ground fields"); 7×7: **250** (Sedoglavic [23], non-commutative, so any ring); 8×8: **336** (= Strassen ⊗ rank-48; mod 2 has $329 = 7 \cdot 47$ — another instance of the field divergence this paper's program probes). What beating each by 1 or 2 buys same-base pipelines at $n = 4096$ (speedup $= (r/(r-k))^{\log_m 4096}$; wall clock × matmul share as above):

base	record r	exponent	-1	-2	overtakes rank-48 at
5×5	93	2.816	+5.7%	+11.9%	≤ 89
6×6	153	2.808	+3.1%	+6.3%	≤ 148
7×7	250	2.837	+1.7%	+3.5%	≤ 229
8×8	336	2.797	+1.2%	+2.4%	≤ 332

Two readings. A -1 is worth more at smaller bases: the recursion depth $\log_m n$ shrinks as m grows, so a 5×5 improvement compounds through 5.2 levels at $n = 4096$ while an 8×8 one gets only 4. And no -1 or -2 at these sizes overtakes the rank-48 4×4 base — the nearest paths to a new best base run through $5\times5 \leq 89$ (-4) and $8\times8 \leq 332$ (-4; especially interesting because today's 336 is *inherited* from rank-48 via Strassen, so any direct 8×8 construction at ≤ 332 would beat every 4×4-blocked pipeline at once).

The additions record — pricing the dual lever. §4's caveat made additions a second lever: they vanish from the constraint count but re-enter twice. (i) *Witness generation* evaluates the scheme's three linear networks; its cost scales directly with the network op count, compounding over recursion exactly like the classical additions recursion. (ii) *Circuit shape*: folding the linear forms keeps constraints at r^d but densifies R1CS rows multiplicatively per level, while materializing intermediates keeps rows sparse at the price of $\approx(\text{ops}/32) \cdot m$ extra *linear* constraints — $\approx 10\times m$ at today's op counts, which is why nobody fully materializes; practice interpolates, and every point on that tradeoff curve scales monotonically with the op count. The record: [10]'s Appendix B states **341 operations** for the rank-48 networks ($L = 104$, $R = 84 + 1$ shift, $P = 119 + 33$ shifts; "PLinOpt generated" [12], via greedy straight-line synthesis, kernel/factorization routes, and Tellegen transposition, after an isotropy transformation rationalized AlphaEvolve's complex scheme to dyadic coefficients) — but the *living* record is better, twice over. The SLPs distributed in the PLinOpt library's data directory [12] (`github.com/jgdumas/plinopt`, snapshot 2026-07-07) improve on the paper in

two steps. First, `data/4x4x4_48_rational_*.slp`, for the identical matrices, checker-verifies in this repository at **315** ($L = 104{+}4$, $R = 75{+}1$, $P = 110{+}21$). Second — and easy to overlook under its name — `data/4x4x4_48_accurate_*.slp`, a *different* rank-48 decomposition distributed for its numerical accuracy, checker-verifies at $L = 80{+}4$, $R = 68{+}8$, $P = 108{+}16 =$ **284 operations** (256 additions + 28 dyadic constant-multiplications), and its matrix triple Brent-verifies over $\mathbb{Q}$ as a true $\langle 4, 4, 4\rangle$ scheme under the same index convention as the rational triple (protocol calibrated on the known-good case). The living record is therefore **284**, 57 operations below the published 341 — the authors improved their own artifact past their paper, and the numerically-accurate variant, not the rational one, is the cheapest. The 284 transfers verbatim to every odd-characteristic $\mathbb{F}_p$: the divisions $/2^k$ are multiplications by the fixed field constants $((p{+}1)/2)^k$, so over a proof field the program is 256 additions plus 28 constant-multiplications — and under the R1CS objective the constant-multiplications fold into linear-combination coefficients for free (Appendix C), leaving density and witness generation as the quantities the 256 adds govern. Over Goldilocks the codegen'd 284-op networks (`bench284r`, exactness gated at $n = 16$ and 64) measure 4–5% faster witness generation than the 315-op networks at $n = 64$ and $n = 1024$ — the expected dilution of a 10% linear-phase cut by the multiplication core. Our own checker-gated searches have **not** improved on the artifact — the best verified alternative orientation reaches 365, sixty signed-permutation and forty dyadic-sandwich orbit variants bottom out at 371 and 422 respectively, an eight-hour CSE storm found nothing below the rational triple's counts, a 4,500-invocation optimizer-portfolio storm converged to exactly the distributed side counts, and — the strongest evidence — **exact window resynthesis** certifies local optimality: every closed sub-DAG window of the three programs up to 12 operations (958 windows; ≤ 8 inputs, ≤ 5 outputs) was posed to an exact branch-and-bound synthesizer over the dyadic move menu, and **876 windows are proven exactly tight at their current cost, zero admit any improvement, and 82 of the largest remain undecided at a 300-second cap** — the 284 is not merely a fixed point of the searches that produced it but locally unimprovable at width 12. An earlier in-repo "329" is on record as a counting artifact caught by output-checker gating (a methodological cautionary tale the repository preserves). **No nontrivial lower bound is known**, so whether 284 is tight globally remains open. What reductions would buy if found: both sinks scale linearly, so a 250-op network would be worth ≈12% and a 200-op network ≈30% *of the adds-driven components* (rebased to 284) (witness generation and the density/materialization tax) — constant-class wins, stacking freely with the exponent-class wins above.

Appendix C. Measured: the predictions on real proving stacks and real machines

We implemented the three gadgets — naive, Strassen 2×2:7, and the rank-48 4×4 recursion with the Brent-verified DPS coefficients — as R1CS circuits over BN254 (arkworks Groth16; `matmul/benchzk/`), and measured; then carried the measurement across proving stacks (a PLONKish/AIR port, `matmul/benchair/`) and execution substrates (CPU scalar, NEON lanes, Apple GPU). The arc in one line: the constraint counts are vindicated exactly; R1CS density is a real tax; AIR arithmetization erases it; and witness generation obeys a substrate law — multiplies, additions, or bytes decide the winning scheme depending on the machine.

Constraint counts land exactly on §3's formulas. $n = 4$: 64 / 49 / **48** multiplication constraints — a 4×4 witness×witness product proven in 48 constraints. $n = 16$: 4096 / 2401 / 2304; $n = 64$: 262,144 / 117,649 / 110,592 ($= n^3$, $7^{\log_2 n}$, $48^{\log_4 n}$). Proofs verify; additions and dyadic scalings cost zero constraints.

Wall clock does not follow constraint count at small n — the density tax is real. Fully folded, the depth-3 rank-48 circuit's rows carry the *compounded* linear-form support, and at $n = 64$ its proving time loses to naive by 17.6× (42.9 s vs 2.5 s) despite 2.37× fewer constraints. The §4 materialization analysis, measured as a schedule (materialize the top t levels' combinations as witness rows, fold below):

levels materialized	constraints	prove (s)
0 (fold all)	114,688	42.9
1	143,360	26.4
2 (optimum)	**229,376**	**12.7**
3 (materialize all)	487,424	13.1

The hybrid optimum recovers 3.4× over naive folding — and still loses to the naive gadget at this size, because Groth16's prover tracks total matrix *nonzeros* more closely than row count. (The materialization row counts match $112 \cdot \sum_\ell 48^{\ell-1}16^{d-\ell}$ exactly.)

At $n = 256$ (depth 4) the density tax becomes an inversion, then a wall. Naive: 16.8M constraints, setup 99 s, prove 119 s. Strassen: 5.83M constraints — 2.9× fewer — setup 433 s, prove 389 s: **3–4× slower on 2.9× fewer constraints**, because at depth 8 its folded rows are dense enough that nonzeros, not rows, set the price. The rank-48 gadget could not be measured at $n = 256$ at *any* materialization level on this 64 GB machine: folded configurations (matlv ≤ 3) balloon past a 190 GB footprint into swap (resident memory reads ∼14 GB while macOS compresses and pages — a measurement hazard in its own right), and materialize-everything (matlv 4) exceeded 46 GB resident during synthesis. A ∼128 GB-class machine is the entry ticket for depth-4 rank-48 Groth16 at bounded density.

The whole pipeline at one scale (benchzk `--full`: witness generation, NTTs at the actual proof-domain size, setup, prove, verify; $n = 64$, six proving threads, background load — the relative structure is the payload, and the idle-machine materialization optimum above reproduces):

gadget	constraints	domain	NTT fwd	setup	prove	verify
naive	266,240	2^{19}	27 ms	1.4 s	1.5 s	0.23 s
Strassen	121,745	2^{17}	6.6 ms	1.5 s	1.4 s	0.23 s
rank-48, matlv 0	114,688	2^{17}	7.3 ms	30.5 s	27.7 s	0.23 s
rank-48, matlv 1	143,360	2^{18}	14 ms	17.2 s	17.0 s	0.23 s
rank-48, matlv 2	229,376	2^{18}	14 ms	7.9 s	8.6 s	0.24 s
rank-48, matlv 3	487,424	2^{19}	26 ms	8.2 s	8.9 s	0.23 s

Strassen at this depth is still density-benign (naive-class prove on 2.2× fewer constraints — its inversion only appears at depth 8); rank-48's fewest-constraint config (matlv 0) is its slowest, and the interior optimum at matlv 2 stands. Witness generation (3.7–4.0 ms) and verification are scheme-independent here, and the NTT column simply tracks domain size — Strassen's smaller domain buys a 4× cheaper transform than naive's, a reminder that constraint *count* still owns the transform even when density owns the MSMs.

The density tax, erased — measured on a PLONKish/AIR stack. The materialization tables above price R1CS's defect: linear combinations live in per-proof constraint rows, so Groth16 charges the rank-48 gadget 17.6× over naive for the *same statement*. An AIR arithmetization moves those combinations into the constraint polynomial's *constants*, where they cost nothing per proof. We ported the gate to Plonky3 uni-stark over BabyBear (Poseidon2, two-adic FRI): two AIRs over the *identical* 48-column trace — 16 A, 16 B, 16 C cells per row, one tile per row, commitment work held constant — one with the naive gate (64 in-constraint multiplies), one with the rank-48 gate, which is the 284-op SLP evaluated over AIR expressions (the same El-generic generated code that runs witness generation over scalars, blocks, and NEON lanes is *literally the gate*; products are never materialized as cells). Both accept the honest trace and reject a tampered product. Measured: naive-gate 16.9 / 56.1 / 224.1 ms at $2^{12}/2^{14}/2^{16}$ tiles; rank-48-gate 16.9 / 56.8 / 230.0 ms — **parity within 3%, against Groth16's 17.6× inversion**. The density tax is an artifact of the MSM cost model, structurally absent from AIR stacks. Honestly stated, at flat tile granularity the rank-48 gate does not *win* either — in-constraint naive materializes nothing, and 48-vs-64 quotient multiplies trade against 284 extra additions over a multiply-cheap field. The multiplication-count advantage pays where operands and products transit committed columns anyway: memory-mediated zkVM precompile chips and block-recursive layouts, where cells track operation count and 48/64 compounds per level.

The row lever, measured. We then built that shape: one parametric mul-schedule AIR — one row per *multiplication*, operands and outputs in committed columns, all scheme wiring (L/R selection, P scatter, group selectors) in preprocessed columns — the honest form of a zkVM precompile's multiply table. Naive and rank-48 are two preprocessed schedules of the *same* 67-column AIR: 64 versus 48 rows per tile. The trace generator Brent-gates the schedule tables (every tile's final accumulator must equal schoolbook C — an

independent field verification of the 284 triple over BabyBear), and both schedules accept honest traces and reject tampered outputs. At equal trace height — fixed prover budget, who packs more tiles — the rank-48 schedule proves 21,845 tiles where naive proves 16,384 in the same 5.75 s at 2^{20} rows: **1.34× throughput, the full** 64/48 **row advantage delivered as wall clock** (1.337× at 2^{18}; theory 1.333×). The two endpoints bracket deployment: materialize nothing and the gate choice is free; materialize per multiplication, as memory-mediated precompile tables do, and rank pays in full — compounding $(48/64)^d$ through recursive schedules. This is the program's thesis reduced to one measured sentence: **in modern proving stacks, rank buys committed rows, and rows are wall clock.**

The memory argument, closed. The mul-schedule chip models memory by replicating operands in committed columns; the deployment-honest question is what a *sound* memory interface costs. We built it: a two-stage prover on Plonky3 primitives (LogUp challenges sampled *after* the main-trace commitment — the ordering without which any memory argument is forgeable — with the permutation columns committed in a second round and folded into the same quotient and opening), and on it a tile-interface precompile chip: a 48-row-per-tile memory band emitting (address, value, multiplicity), a compute band whose first 48 rows each carry one memory operation (32 reads + 16 writes per tile, identical for both schedules — the SP1/RISC-Zero precompile shape), and a running LogUp accumulator whose last-row vanishing *is* memory consistency. Soundness is tested, not argued: all four forgery classes — corrupted memory value, a compute read diverging from the committed image, inflated multiplicity, corrupted output write — are rejected; honest traces accept for both schedules. Measured at equal trace height: **1.176× at 2^{14} and 1.163× at 2^{16} rank-48 throughput over naive, on the row-count prediction** $112/96 = 1.167\times$ (each tile pays 48 memory rows regardless of schedule, so the flat-schedule 1.33× dilutes to ∼1.17×). The deployment bracket is now measured at all three points: materialize nothing → the gate is free (parity); memory-mediated with a sound argument → **1.17**×, with 1.34× as the replicated-operand ceiling; one row per operation → the multiplication advantage drowns. A rank-48 precompile ships with a ∼17% sound throughput edge on the identical memory interface — and every further row saved by lower rank compounds through the same arithmetic.

Witness generation is friendlier territory. The prover must also *compute* the witness — every intermediate product, in the clear, over the field — and here the same networks measure very differently (bench315/bench315r; the 315-op PLinOpt SLPs codegen'd to Goldilocks, exactness gated at every size — regenerating from the 284-op accurate triple (Appendix B.5) improves these rank-48 times a further 4–5% at $n = 64$ and 1024 in back-to-back A/B):

n	naive	blocked naive	rank-48	ratio vs blocked
16	0.019 ms	0.019 ms	0.035 ms	1.83
64	1.15 ms	1.15 ms	0.926 ms	**0.80**
256	80.7 ms	79.9 ms	47.0 ms	0.59
1024	6.30 s	5.08 s	2.32 s	**0.46**
4096	936 s	246 s	83.0 s	**0.34**

(Arithmetic tuned to the field's design: division-free Goldilocks reduction via $2^{64} \equiv 2^{32}-1$, halving as shift-plus-fixup, doubling as one modular add, `target-cpu=native` + LTO — ≈2.7× over the first-cut build; a cache-blocked naive is the control, worth 20% at $n = 1024$.) The crossover sits at $n \approx 64$ (hybrid cutoff: recurse to 16×16 tiles, naive below), and by $n = 1024$ the recursion is 2.2× faster than the blocked control — the measured ratio 0.46 sitting just above the pure multiplication-tree limit $(48/64)^3 = 0.42$, i.e. the 289-add linear phases are nearly fully amortized. Adds are cheap CPU operations, so witness generation inherits the exponent advantage at small n, long before the commitment-side density crossover. At $n = 4096$ (measured once, under background load) the ratio reaches **0.34** against the blocked control — almost exactly the compounding prediction $0.46 \times (48/64) = 0.345$ — and 11.3× over plain naive; the 284-vs-315 A/B is within noise at this size.

Machine-level witness generation: mult–add fusion, calibrated. The $\mathbb{F}_p$ analogue of fused multiply–add is *delayed reduction*: accumulate 128-bit products as (lo, hi) pairs of 64-bit sums — two plain adds per term — and reduce once per output. Microbenchmarks on this machine (dependent-chain methodology, correctness gated on 10^5 random dot products) price the scalar Goldilocks multiply at 9.3 ns, a deferred multiply-

accumulate at 1.25 ns, the final combine-and-reduce at 10.4 ns, and halvings/adds at 1.2–1.5 ns: deferral is ∼2.7× cheaper, and reductions ∼1.7× dearer, than naive op counting assumes. Under these measured constants a cost model over the rank-48 linear networks (shift-aware, deferral-aware; `machinecost.py`) *appeared to reorder the record book*: the 284-op accurate triple — whose every output path crosses a halving, blocking deferral — modeled at 753 cycles per 4×4 tile, while a 357-op triple from this repository's own optimizer, with a fully deferrable product side, modeled at **705**. Three storm searches over exponent-relabel moves confirmed the 284's blockers are structural, and an 18-variant orientation×gauge sweep under the calibrated score left the 705 incumbent standing. **Then the measured implementation refuted the model.** We codegen'd the deferrable network with true delayed reduction — products kept as unreduced (lo, hi) pairs, u128 limb-sum accumulation, subtraction by bound-tracked negation constants, one combine per output — field-gated it against schoolbook on 20,000 random tiles, and timed all paths (`bench705`): 284-scalar 282 ns/tile, ours-scalar 331 ns, ours-*delayed* **461 ns** — 1.63× *slower* where the model said 0.94×. The diagnosis is instructive: dependent-chain microbenchmarks price *latency*, but a 4×4 tile executes at *throughput* — the out-of-order core overlaps 48 independent multiply-reduce chains, hiding exactly the reduction latency that deferral skips, while the delayed path forfeits that parallelism to longer dependency chains and roughly doubled live register state. At tile granularity, machine-optimal ≈ operation-minimal after all: **the 284 is the measured-fastest rank-48 witness-generation path**, and delayed reduction stays where it is already standard — long, all-positive dot products (it gates correct and wins there), i.e. naive inner loops, not rank-48 product networks. A cost model earns trust only against silicon; this one paid for its lesson in one afternoon.

The SIMD lever that survives is lane-batching across tiles — throughput-shaped by construction, so the refutation above does not touch it — and over a 31-bit field it is large. BabyBear ($p = 2^{31}-2^{27}+1$, Montgomery form) with an explicit NEON 4-lane Montgomery multiply (widening `vmull` pairs; the same El-generic 284 networks run unchanged over scalars and 4-lane batches; all paths field-gated per lane against scalar):

path	scalar	4-lane NEON	lane speedup
BabyBear naive tile	53.5 ns	**20.0 ns**	2.68×
BabyBear 284 tile	177.4 ns	76.8 ns	2.32×

Auto-vectorization alone bought only 1.47× on the multiply-bound naive path — compilers will not synthesize widening Montgomery chains; the intrinsics nearly double it. Cross-field, a BabyBear-NEON naive tile runs **9.9× faster than a Goldilocks-scalar naive tile** (the 284 path 3.7×). And one structural economics shift rides along: with 32-bit multiplies this cheap, additions dominate the tile — naive (64 muls, 48 adds) beats the 284 (48 muls, 256 adds) by 3.8× at tile level — so the rank-48 recursion crossover is *field-dependent* and sits later over BabyBear than over Goldilocks. Field choice moves not just the transform (measured above) but which matmul algorithm wins witness generation at which scale. And the *substrate* moves it again: on the Apple M4 Pro GPU (Metal compute, the 284-op SLP lowered to a shader — same generated program, sixth target; every kernel gated bit-for-bit against the CPU reference), naive and rank-48 tile kernels converge to ∼0.9 **ns/tile regardless of algorithm** — 213 GB/s of the chip's ∼273 GB/s, memory-bandwidth bound, 22× the NEON-naive rate. (An apparent rank-48 win on first measurement was a compilation artifact — loop-indexed thread arrays spilling to local memory — killed by an unrolled-naive control.) Three substrates, three scarce resources, one law: CPU-Goldilocks rations multiplies, CPU-BabyBear rations additions, the GPU rations bytes; witness-generation algorithm choice must follow the machine's binding constraint, and on GPUs the mult-count question is moot at tile granularity — which converges with the AIR port's deployment conclusion: reduce committed cells and rows, not arithmetic. The recursion bench makes the field dependence quantitative (scalar Montgomery, methodology matched to the Goldilocks table above, gates at every size): over BabyBear the rank-48-vs-blocked ratio runs 1.61 at $n = 64$, 1.31 at 256, and crosses at **0.97 at** $n = 1024$ — the crossover sits at $n \approx 10^3$ over BabyBear versus $n \approx 64$ over Goldilocks, two full recursion levels later. One program, two honest summaries: over a 64-bit field, low rank pays from $n \approx 64$ and reaches 3× by $n = 4096$; over a 31-bit SIMD-friendly field, the same networks only break even around $n \approx 10^3$ — and the NTT table above shows where the 31-bit field spends the advantage instead.

Reading. The exponent advantage is real and the counts are exactly as predicted; the *crossover* where it beats naive wall clock on Groth16/BN254 sits beyond $n = 64$ at bounded density (constraint ratio 0.85 at

n=64 → 0.67 at 256 → 0.53 at 1024, against a ≈2 to 3× density constant), i.e. likely $n \gtrsim 10^3$ — LLM-scale matrices, per §3, not toy benchmarks. Three levers move the crossover in: PLONKish/AIR custom gates that hard-code the 48 fixed combinations as constraint constants (density becomes free — now *measured*: the Plonky3 gate above proves at parity where Groth16 charged 17.6×); STARK/AIR stacks whose cost model is hashing-dominated rather than nonzero-dominated; and lower-adds networks (B.5), which shrink the density constant directly — the measured reason the adds record matters. Reproducible from the repository (`benchzk`, `benchair`, fixed seeds).

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the engines and drafted this text under the author's direction and review.

References

[1] V. Strassen. *Gaussian elimination is not optimal.* Numer. Math. 13:354 to 356, 1969.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Scalable, transparent, and post-quantum secure computational integrity.* ePrint 2018/046.

[3] A. Fawzi et al. *Discovering faster matrix multiplication algorithms with reinforcement learning.* Nature 610:47 to 53, 2022.

[4] J. Thaler. *Time-optimal interactive proofs for circuit evaluation.* CRYPTO 2013.

[5] A. Gabizon, Z. Williamson, O. Ciobotaru. *PLONK.* ePrint 2019/953.

[6] J. Groth. *On the size of pairing-based non-interactive arguments.* EUROCRYPT 2016.

[7] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles.* STOC 2008.

[8] U. Haböck, D. Levit, S. Papini. *Circle STARKs.* ePrint 2024/278.

[9] T. Liu, X. Xie, Y. Zhang. *zkCNN.* CCS 2021.

[10] J.-G. Dumas, C. Pernet, A. Sedoglavic. *A non-commutative algorithm for multiplying* 4×4 *matrices using 48 non-complex multiplications.* arXiv:2506.13242 (2025). (The 341-operation SLP accounting is its Appendix B; scheme and SLPs distributed via PLinOpt's data directory [12].)

[11] A. Novikov et al. *AlphaEvolve: a coding agent for scientific and algorithmic discovery.* 2025, arXiv:2506.13131. (Origin of the rank-48 scheme, over $\mathbb{C}$; no operation counts there.)

[12] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic. *PLinOpt, a library for optimizing linear programs.* github.com/jgdumas/plinopt.

[13] EZKL. github.com/zkonduit/ezkl.

[14] Polygon Zero. *Plonky2.* Whitepaper, 2022.

[15] RISC Zero. *zkVM documentation*, 2023.

[16] A. Kothapalli, S. Setty, I. Tzialla. *Nova.* CRYPTO 2022.

[17] M. Kauers, J. Moosbauer. *Flip graphs for matrix multiplication.* arXiv:2212.01175.

[18] G. Grassi et al. *Poseidon.* USENIX Security 2021.

[19] G. Sidebottom. *A 55-addition rank-23 scheme for* 3×3 *matrix multiplication.* This repository, 2026. DOI 10.5281/zenodo.21240904.

[20] G. Sidebottom. *Rigidity of the rank-48* 4×4 *scheme* and *The Flower.* Companion notes, this repository, 2026.

[21] R. Gennaro, C. Gentry, B. Parno, M. Raykova. *Quadratic span programs and succinct NIZKs without PCPs.* EUROCRYPT 2013. (The QAP compilation.)

[22] J. Moosbauer, M. Poole. *Flip graphs with symmetry and new matrix multiplication schemes.* ISSAC 2025; arXiv:2502.04514. (5×5:93 and 6×6:153, over arbitrary ground fields.)

[23] A. Sedoglavic. *A non-commutative algorithm for multiplying* 7×7 *matrices using 250 multiplications.* Preprint, 2017.

[24] A. Rosowski. *Fast commutative matrix algorithms.* 2019, arXiv:1904.07683. (3×3 in 21 commutative products; odd-n tiles in $n(n^2 + 2n - 1)/2$.)

[25] *Exploring commutative matrix multiplication schemes via flip graphs.* 2025, arXiv:2506.22113.